Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers 3 Marks

Bloom's Level: K4 – Analyse

Organise your classes into packages and protect the data of each class.

Discuss:

- Your package structure (for example: parking.model, parking.service, parking.app)
- The access modifier chosen for each field and method (private, default, protected, public) and why
- How getters and setters validate data (for example: rejecting a negative duration or an invalid slot number)
- Where wrapper classes and autoboxing are used, and any risk they introduce (for example: comparing Integer objects with ==)

Alternative Concept and Complexity Analysis

Your fields are private with validating setters. If the fields were made public and validation were done by the calling code, what would the solution be and what could go wrong? Analyse the time cost of validation per update and the number of places where the same check would have to be repeated.

Evidence Expected

A package structure, justified access modifiers and validated getters and setters that protect the object state.

Solution:

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers

Campus Smart-Parking System – 3 Marks

This solution continues the same Campus Smart-Parking System from Tasks 6–9.

We will organise the Java classes into separate packages, protect data using access modifiers and validate values through getters and setters.

1. Introduction to Encapsulation

Encapsulation is an object-oriented programming concept that combines data and methods into a single class and protects the data from direct access.

In Java, encapsulation is achieved using:

- Private data members.
- Public getters and setters.
- Access modifiers.
- Data validation.

In the Campus Smart-Parking System, vehicle registration numbers, owner names and parking slot numbers are protected from invalid modifications.

2. Package Structure

The project is organised into three main packages.

Task10

src

parking.model

Vehicle.java

TwoWheeler.java

Car.java

ElectricCar.java

Chargeable.java

ParkingSlot.java

parking.service

ParkingService.java

parking.app

Main.java

out (compiled files generated automatically)

Purpose of each package:

- parking.model – Contains vehicle and parking slot classes.
- parking.service – Contains parking fee calculation and validation logic.
- parking.app – Contains the main application program.

3. Access Modifiers

Java provides four main access levels.

Modifier	Purpose	Usage
private	Accessible only within the same class	Vehicle fields
default	Accessible within the same package	Internal helper methods
protected	Accessible within the package and subclasses	ratePerHour()
public	Accessible from other packages	Getters and setters

Justification:

- Fields are declared private to prevent direct modification.
- Getters and setters are public to allow controlled access.
- The ratePerHour() method is protected because subclasses provide their own implementation.
- Default access is suitable for internal methods that do not need to be accessed outside their package.

4. Getters, Setters and Data Validation

Getters return private field values, while setters modify them after checking whether the supplied values are valid.

Examples:

- Owner name must not be empty.

- Parking slot number must be positive.
- Parking duration must not be negative.
- Null values must be checked before unboxing wrapper objects.

Example:

```
public void setSlotNumber(Integer slotNumber) {
   if (slotNumber == null || slotNumber <= 0) {
      throw new IllegalArgumentException(
         "Invalid slot number");
   }
   this.slotNumber = slotNumber;
}
```

This prevents invalid data from entering the object.

5. Wrapper Classes and Autoboxing

Wrapper classes convert primitive data types into objects.

For example:

- int → Integer
- double → Double
- boolean → Boolean

In this system, Integer is used to store parking slot numbers and parking durations.

Autoboxing: Automatically converts a primitive value into its wrapper object.

```
Integer slotNumber = 101;
```

Unboxing: Automatically converts a wrapper object into a primitive value.

```
int slot = slotNumber;
```

Risk: Comparing two Integer objects using == may compare object references instead of their values.

```
Integer a = 1000;
Integer b = 1000;
System.out.println(a.equals(b)); // true
```

Therefore, equals() should be used to compare wrapper object values.

6. Complete Java Program – File by File

Create the folders and files exactly as shown above.

Important: Each code block below belongs in a separate .java file. Copy only the Java code, not the code-block markers.

File 1: Vehicle.java

Path: src/parking/model/Vehicle.java

```
package parking.model;

public abstract class Vehicle {

    private final String regNo;

    private String ownerName;

    public Vehicle(String regNo, String ownerName) {

        if (regNo == null || regNo.isBlank()) {

            throw new IllegalArgumentException(

                "Invalid registration number");

        }

        this.regNo = regNo;

        setOwnerName(ownerName);

    }


    public String getRegNo() {

        return regNo;

    }


    public String getOwnerName() {

        return ownerName;

    }


    public void setOwnerName(String ownerName) {

        if (ownerName == null || ownerName.isBlank()) {
```

```
            throw new IllegalArgumentException(
                "Owner name cannot be empty");
        }

        this.ownerName = ownerName;
    }

    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) {
            throw new IllegalArgumentException(
                "Duration cannot be negative");
        }

        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
}
```

File 2: TwoWheeler.java

Path: src/parking/model/TwoWheeler.java

```
package parking.model;

public class TwoWheeler extends Vehicle {

    public TwoWheeler(String regNo, String ownerName) {
        super(regNo, ownerName);
    }
```

```
    @Override
    protected double ratePerHour() {
        return 10;
    }
}
```

File 3: Car.java

Path: src/parking/model/Car.java

```
package parking.model;


public class Car extends Vehicle {


    public Car(String regNo, String ownerName) {
        super(regNo, ownerName);
    }


    @Override
    protected double ratePerHour() {
        return 30;
    }
}
```

File 4: Chargeable.java

Path: src/parking/model/Chargeable.java

```
package parking.model;


public interface Chargeable {


    double chargingFee(int kWh);
```

```
}
```

File 5: ElectricCar.java

Path: src/parking/model/ElectricCar.java

```
package parking.model;
public class ElectricCar extends Car implements Chargeable {
    public ElectricCar(String regNo, String ownerName) {
        super(regNo, ownerName);
    }
    @Override
    public double chargingFee(int kWh) {
        if (kWh < 0) {
            throw new IllegalArgumentException(
                "Invalid charging units");
        }
        return kWh * 12;
    }
}
```

File 6: ParkingSlot.java

Path: src/parking/model/ParkingSlot.java

```
package parking.model;


public class ParkingSlot {


    private Integer slotNumber;
    private boolean occupied;


    public ParkingSlot(Integer slotNumber) {
        setSlotNumber(slotNumber);
```

```
        this.occupied = false;
    }

    public Integer getSlotNumber() {
        return slotNumber;
    }

    public void setSlotNumber(Integer slotNumber) {
        if (slotNumber == null || slotNumber <= 0) {
            throw new IllegalArgumentException(
                "Invalid parking slot number");
        }

        this.slotNumber = slotNumber;
    }

    public boolean isOccupied() {
        return occupied;
    }

    public void setOccupied(boolean occupied) {
        this.occupied = occupied;
    }
}
```

File 7: ParkingService.java

Path: src/parking/service/ParkingService.java

```
package parking.service;
```

```
import parking.model.Vehicle;


public class ParkingService {


    public double calculateParkingFee(
            Vehicle vehicle, Integer minutes) {


        if (vehicle == null) {
            throw new IllegalArgumentException(
                "Vehicle cannot be null");
        }


        if (minutes == null || minutes < 0) {
            throw new IllegalArgumentException(
                "Invalid parking duration");
        }


        return vehicle.calculateFee(minutes);
    }
}
```

File 8: Main.java

Path: src/parking/app/Main.java

```
package parking.app;


import parking.model.*;
import parking.service.ParkingService;
public class Main {
    public static void main(String[] args) {
```

```
ParkingService service = new ParkingService();
Vehicle v1 = new TwoWheeler(
    "TN01A1", "Arun");
Vehicle v2 = new Car(
    "TN02B2", "Priya");
ElectricCar v3 = new ElectricCar(
    "TN03C3", "Kumar");
ParkingSlot slot = new ParkingSlot(101);
Integer duration = 130;
System.out.println("Campus Smart Parking System");
System.out.println("\nVehicle Details:");
System.out.println("Registration: " + v1.getRegNo());
System.out.println("Owner: " + v1.getOwnerName());
System.out.println("Registration: " + v2.getRegNo());
System.out.println("Owner: " + v2.getOwnerName());
System.out.println("Registration: " + v3.getRegNo());
System.out.println("Owner: " + v3.getOwnerName());
System.out.println("\nParking Fees:");
System.out.println("TwoWheeler: Rs " +
    service.calculateParkingFee(v1, duration));
System.out.println("Car: Rs " +
    service.calculateParkingFee(v2, duration));
System.out.println("ElectricCar: Rs " +
    service.calculateParkingFee(v3, duration));
System.out.println("\nParking Slot:");
System.out.println("Slot Number: " +
    slot.getSlotNumber());
slot.setOccupied(true);
```

```
        System.out.println("Occupied: " +
            slot.isOccupied());
        System.out.println("\nCharging Fee:");
        System.out.println("ElectricCar: Rs " +
            v3.chargingFee(10));
        System.out.println("\nValidation:");
        System.out.println("Duration: " + duration + " minutes");
        System.out.println("Slot Number: " +
            slot.getSlotNumber());
        System.out.println("Integer comparison: " +
            Integer.valueOf(1000).equals(
                Integer.valueOf(1000)));
    }
}
```

7. Expected Output

```
Campus Smart Parking System
Vehicle Details:
Registration: TN01A1
Owner: Arun
Registration: TN02B2
Owner: Priya
Registration: TN03C3
Owner: Kumar
Parking Fees:
TwoWheeler: Rs 30.0
Car: Rs 90.0
ElectricCar: Rs 90.0
```

Parking Slot:

Slot Number: 101

Occupied: true

Charging Fee:

ElectricCar: Rs 120.0

Validation:

Duration: 130 minutes

Slot Number: 101

Integer comparison: true

8. Alternative Concept – Public Fields vs Private Fields

If all fields were declared public instead of private, other classes could directly modify the data without using validating setters.

Example:

```
class ParkingSlot {
   public int slotNumber;
   public boolean occupied;
}
```

A calling class could assign an invalid value:

```
ParkingSlot slot = new ParkingSlot();
slot.slotNumber = -10;
```

This creates several problems:

- Invalid parking slot numbers can enter the system.
- Different classes may modify data incorrectly.
- Validation logic must be repeated in multiple places.
- Maintaining and debugging the application becomes difficult.

Comparison

Factor	Encapsulation	Public Fields
Data access	Controlled	Unrestricted
Validation	Inside setters	Calling code
Security	Better protection	Less protection
Maintenance	Easier	Difficult
Invalid data	Prevented through validation	Can enter easily

Time Complexity Analysis

- Encapsulation: Each validation takes O(1) time per update.
- Public fields: Each validation also takes O(1) time, but the same checks must be repeated wherever the fields are modified.
- If there are n different update locations, validation logic may need to be maintained in all n locations.

Therefore, encapsulation reduces code duplication and improves data reliability without changing the basic time complexity of validation.

Conclusion: Private fields with validated getters and setters are more suitable for the Campus Smart-Parking System because they protect object data, prevent invalid values and improve maintainability.

